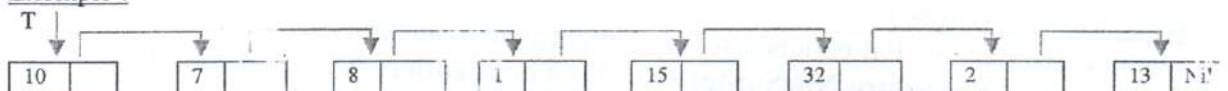


Examen Semestre 2		
Matière : Algorithmique Et Structures de Données 2		
Enseignantes : Grira H, Gribaa N, Hila K, Rekik S, Marsit I, Masmoudi M		Date : Mai 2024
Niveau	1 ^{ère} Année	Durée : 01H30
Filière	GLSI & LISI	Documents : non autorisés
Barème approximatif	8 pts + 6 pts + 6 pts	Nombre de pages : 2

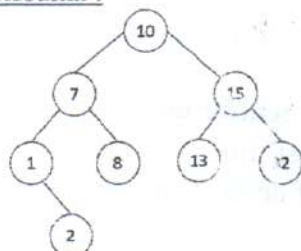
Exercice 1 (8 points)

- Définir la structure 'Nœud' qui permet d'implémenter le nœud d'un arbre binaire contenant une valeur entière.
- On suppose qu'on dispose de la procédure **AjoutNoeudABR** (var R, NN : ArbreB) qui permet d'insérer le nœud pointé par NN dans l'arbre de recherche de racine R. NN contient une valeur de type entier et les constantes NIL pour adresses de fils gauche et fils de droite.
Ecrire l'algorithme de la procédure **CreateArbreBR** (T : Pliste ; var R : ArbreB) qui permet de créer un arbre binaire de recherche de Racine R contenant une copie des entiers de la liste chaînée simple de Tête T.

Exemple :



Résultat :



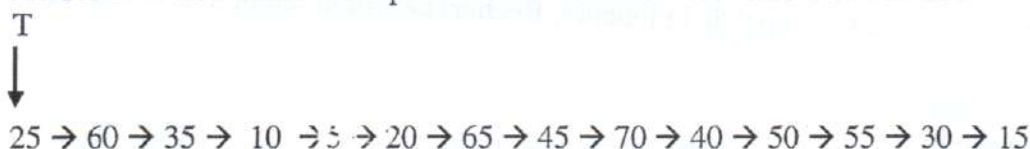
- Ecrire l'algorithme de la procédure **DisplayOrdered** (R : ArbreB) qui permet d'afficher les nœuds de l'arbre binaire de recherche dans un ordre croissant.

Exemple :

Pour l'arbre obtenu dans la question précédente, la procédure **DisplayOrdered** affiche :

1 2 7 8 10 13 15 32

- On vous donne la liste chaînée simple de tête T contenant les valeurs suivantes :



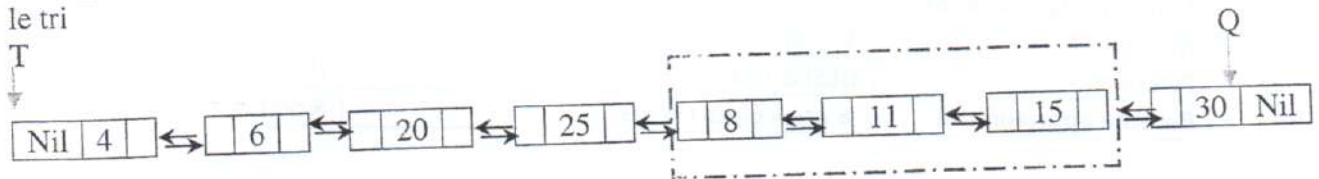
T.A.F

- Dessiner l'arbre binaire de recherche obtenu suite à l'appel à la procédure **CreateArbreBR** par l'insertion des valeurs de la liste de tête T.
- Dessiner les arbres obtenus suite aux suppressions successives et cumulées de ces valeurs : 20, 45. Expliquez pour chaque valeur supprimée comment vous avez effectué la suppression.

Exercice 2 (6 points)

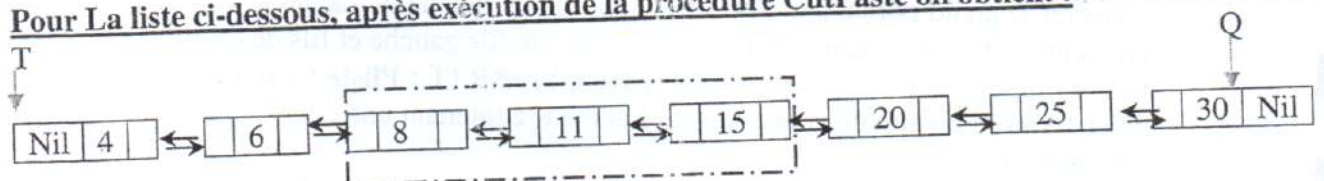
On suppose qu'on dispose d'une liste chaînée double d'entiers et qu'on a lancé un processus de tri dans l'ordre croissant des éléments de cette liste. A la fin de ce processus nous avons obtenu une liste qui est partiellement triée, c'est-à-dire qu'elle contient une séquence d'entiers qu'on doit déplacer pour finaliser le Tri.

Exemple : pour la liste ci-dessous, il suffit de déplacer les éléments dans le cadre pointillé pour finaliser le tri



Ecrire l'algorithme de la **Procédure CutPaste** (var T, var Q : PLBi) qui permet de réaliser la finalisation du processus de tri par déplacement de la séquence mal classée vers l'endroit approprié. On suppose que la liste contient une seule séquence à déplacer.

Pour La liste ci-dessous, après exécution de la procédure CutPaste on obtient :



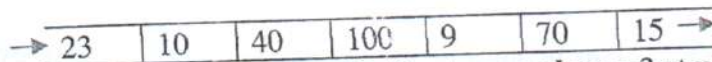
Exercice 3 (6 points)

Soit une file d'attente F d'entiers distincts contenant au moins trois entiers.

1. Ecrire une **Procédure NbrSupInf** (var F : File d'entier; X : entier, var nbsup, var nbinf : entier) qui retourne dans nbsup le nombre d'éléments dans F qui sont supérieur à X, et dans nbinf le nombre d'éléments dans F qui sont inférieur à X.
2. Ecrire une **fonction ValPos**(var F : file d'entier; pos : entier) : entier qui retourne l'entier de la file F dont la position est pos
3. Ecrire une **Fonction RechercheMilieu** (var F : File d'entier) : entier qui retourne la valeur qui vient au milieu (ou presque) de la file d'attente lorsque la file est triée. Vous pouvez utiliser dans votre solution les deux modules des questions 1 et 2. L'objectif de la solution n'est pas de trier la file mais de chercher cette valeur au milieu.

Exemple 1 :

La file F



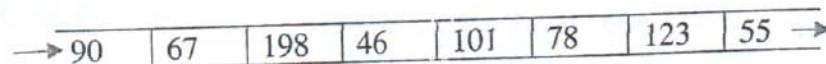
Pour cette file F, la procédure **NbrSupInf** pour X=23 retourne nbsup=3 et nbinf=3

Pour cette file F, l'appel de la fonction **ValPos** pour pos=7 retourne la valeur 23

Pour cette file F, l'appel de la fonction **RechercheMilieu** retourne la valeur 23

Exemple 2 :

La file F



Pour cette file F, la procédure **NbrSupInf** pour X=78 retourne nbsup=4 et nbinf=3

Pour cette file F, l'appel de la fonction **ValPos** pour pos=3 retourne la valeur 78

Pour cette file F, l'appel de la fonction **RechercheMilieu** retourne la valeur 78

N.B

- Faites attention à ne pas perdre le contenu de la file d'attente dans vos solutions
- On suppose l'existence des primitives vues en cours (Enfiler(), Défiler(), FileVide())

Bon Travail